

Technical Specification: Scalpel v1.0.0

Target Builder: AI Coding Agent (Claude Code / Codex) Version: 1.0.0 (Kernel Surgeon Edition) Date: 2025-11-21 Theme: "Surgical Precision for the Linux Kernel"

1. Project Manifesto

Scalpel is a terminal-native, agentic CLI that brings surgical precision to Linux kernel observability and modification. It bridges the gap between high-level operational intent ("Fix the packet drop in TCP") and low-level implementation (calculating hex offsets, writing eBPF bytecode).

Core Philosophy:

- No Recompilation:** All instrumentation is dynamic (eBPF/kprobe/uprobe).
- Safety First:** The BPF Verifier is the ultimate gatekeeper. Unsafe code is rejected before loading.
- Semantic Addressing:** Users address logic ("the checksum error block"), not memory addresses (`0xff...`).
- Zero Overhead (When Dormant):** Hooks are JIT-compiled and only incur cost when active.

2. Technology Stack

- Runtime:** Rust (for safety and C-interop) or C++20 (if integrating heavy LLVM libs).
Recommendation: Rust due to strong eBPF ecosystem (`aya-rs`, `libbpf-rs`).
- eBPF Framework:** `libbpf` (C) with CO-RE (Compile Once - Run Everywhere).
- Debug Info Parser:** `gimli` (Rust) or `libdw` (C) for DWARF parsing.
- Disassembler:** `capstone` for runtime instruction analysis.
- LLM Backend:** Anthropic Claude 3.5 Sonnet (via Bedrock/API) for logic synthesis.
- TUI:** `ratatui` (Rust) for the "Claude Code" aesthetic.

3. Architecture: The "Surgical Loop"

The application operates as a state machine with four distinct phases:

Phase 1: The Diagnostic (Analysis)

- Input:** User prompt ("Hook `tcp_v4_rcv` where it drops packets").
- Action:**
 - Symbol Resolution:** Locate `tcp_v4_rcv` in `/proc/kallsyms`.
 - DWARF Parsing:** Load `vmlinux` (or debug symbols).
 - Semantic Mapping:** The LLM analyzes the source/disassembly to map "drops packets" to a specific basic block (e.g., `JNE` instruction pointing to an error label).
- Output:** A precise Instruction Offset (e.g., `tcp_v4_rcv + 0x4a`).

Phase 2: The Prescription (Code Gen)

- Action:**

1. The Agent writes a C program using `libbpf`.
2. **Context Awareness:** It automatically includes the correct kernel headers (`vmlinux.h`) generated from the running kernel.
3. **Register Mapping:** It determines which register holds relevant variables (e.g., "Source IP is in `%rsi`") at that specific offset.

Phase 3: The Procedure (Injection)

- **Action:**
 1. **Verification:** Compile the BPF program and run it against a verifier simulation (if possible) or attempt load.
 2. **Attachment:** Use `perf_event_open` to attach a `kprobe` at the calculated offset.
 3. **Safety Guard:** If the hook is a "Mutation" (write), prompt for explicit user confirmation.

Phase 4: The Monitor (Telemetry)

- **Action:**
 1. Stream events from the BPF Ring Buffer.
 2. Format binary structs into human-readable logs in the TUI.

4. CLI & TUI Specification

4.1 The Interactive REPL

Entry point: `scalpel` (Stateful session).

UI Components:

- `<VitalsPanel />` : Real-time kernel metrics (Load, Interrupts/sec).
- `<DisassemblyView />` : A scrolling pane showing the target function's assembly, highlighting the proposed hook point.
- `<ProbeStatus />` : Active probes list (e.g., `kprobe:tcp_v4_rcv+0x4a [ACTIVE]`).
- `<LogStream />` : The "Output" pane for `bpf_trace_printk` or ring buffer data.

4.2 Core Capabilities (The Tools)

A. `dynamic_instrumentation`

- **Trigger:** "Trace function X", "Inspect variable Y".
- **Logic:**
 - Locate function symbol.
 - If specific logic is requested ("when size > 100"), calculate offset.
 - Generate BPF code to read register/stack.

B. `state_mutation` (The "Knife")

- **Trigger:** "Modify", "Overwrite", "Change return value".

- **Logic:**
 - **Critical Safety:** Requires `bpf_probe_write_user` or `bpf_override_return`.
 - **Constraint:** Only allowed if `CONFIG_BPF_KPROBE_OVERRIDE` is enabled (for return overrides).
 - **UI:** Must display a red "DANGER" modal before execution.

C. `userspace_shim`

- **Trigger:** "Trace malloc in libc", "Hook SSL_write in openssl".
- **Logic:**
 - Resolve library path (`ldconfig -p`).
 - Calculate offset within the `.so` file.
 - Attach `uprobe` / `uretprobe`.

5. Data Structures & Schemas

5.1 The Operation Plan (JSON)

The LLM outputs this structure to propose an action:

```
{
  "intent": "Throttle write operations on ext4",
  "target": {
    "symbol": "ext4_file_write_iter",
    "module": "kernel",
    "semantic_location": "entry"
  },
  "implementation": {
    "type": "kprobe",
    "code_snippet": "int kprobe__ext4(struct pt_regs *ctx) { ... }",
    "is_mutation": true,
    "registers_used": ["RDI", "RSI"]
  },
  "risks": [
    "Modifying write count may corrupt file data if application expects atomicity."
  ]
}
```